

TUGAS 7
Laporan Tugas Pembelajaran Mesin



Disusun oleh:

187241115 | Muhammad Naufal Zaki

S1 SISTEM INFORMASI
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS AIRLANGGA
2026

1. IMPLEMENTASI DECISION TREE CLASSIFIER

Implementasi algoritma *Decision Tree* adalah proses membangun model prediksi yang memetakan fitur-fitur observasi ke dalam kesimpulan nilai target berdasarkan aturan keputusan (aturan *If-Then*). Dalam dokumen ini, metode yang digunakan mencakup seluruh *pipeline* mulai dari pemuatan data, pemodelan, hingga visualisasi arsitektur keputusan.

1.1 Inisialisasi dan Loading Data

Kode diawali dengan mengimpor *library* yang dibutuhkan untuk manipulasi data dan komputasi *machine learning*, di antaranya `pandas` untuk pengelolaan *dataframe*, `matplotlib` untuk visualisasi, serta modul `sklearn.tree` untuk membangun dan memvisualisasikan model *Decision Tree*.

```
# 2. MENAMPILKAN SEMUA DATA
#membaca data
dataframe = pd.read_excel('BlaBla.xlsx')

data = dataframe[['A', 'B', 'C',
                  'D', 'E', 'F',
                  'G', 'H',
                  'I', 'J',
                  'K', 'L',
                  'M', 'N']]

print("data awal".center(75, "="))
print(data)
print("=" * 75)
```

Dataset dimuat dari file berekstensi Excel. Setelah dataset berhasil dimuat ke dalam memori, dilakukan seleksi kolom secara eksplisit (kolom A hingga N). Hal ini berfungsi sebagai bentuk awal dari manajemen memori, memastikan hanya fitur-fitur yang memiliki nilai analitik yang akan diproses lebih lanjut oleh sistem.

1.2 Preprocessing dan Grouping Variabel

Karena dataset ini akan digunakan untuk klasifikasi terarah (*supervised learning*), data harus dipisahkan menjadi dua kelompok utama: matriks fitur dan vektor target.

```
# 3. GROUPING
#grouping yang dibagi menjadi dua
print("GROUPING VARIABEL".center(75, "="))
X = data.iloc[:, 0:13].values
y = data.iloc[:, 13].values
print("data variabel".center(75, "="))
print(X)
print("data kelas".center(75, "="))
print(y)
print("=" * 75)
```

Menggunakan metode `iloc`, 13 kolom pertama dialokasikan sebagai variabel independen x (fitur input), sedangkan kolom ke-14 diisolasi sebagai variabel dependen y (label kelas/target).

1.3 Pembagian Data Pelatihan dan Pengujian

Proses pemisahan data dilakukan untuk menguji kemampuan generalisasi model terhadap data baru yang belum pernah dilihat sebelumnya.

```
# 4. TRAINING DAN TESTING
#pembagian training dan testing
print("SPLITTING DATA 20-80".center(75, "="))
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=0)
print("instance variabel data training".center(75, "="))
print(X_train)
print("instance kelas data training".center(75, "="))
print(y_train)
print("instance variabel data testing".center(75, "="))
print(X_test)
print("instance kelas data testing".center(75, "="))
print(y_test)
print("=" * 75)
print()
```

Fungsi `train_test_split` membagi dataset dengan proporsi 80% untuk data latih ($X_{\text{train}}, y_{\text{train}}$) dan 20% untuk data uji ($X_{\text{test}}, y_{\text{test}}$).

Parameter `random_state=0` diatur untuk memastikan pemisahan data bersifat deterministik dan dapat direproduksi secara konsisten pada eksekusi berikutnya.

1.4 Pemodelan dan Prediksi

Setelah data siap, algoritma pembelajaran mesin diinisialisasi dan dilatih menggunakan set data pelatihan.

```
#pemodelan decision tree
decision_tree = DecisionTreeClassifier(random_state=0)
decision_tree.fit(X_train, y_train)
```

Objek `DecisionTreeClassifier` mempelajari korelasi matematis antara fitur dan target melalui metode `.fit()`. Model yang telah dilatih kemudian digunakan untuk memprediksi kelas dari data uji X_{test} , dan hasilnya disimpan dalam variabel Y_{pred} .

1.5 Evaluasi Performa Model

Untuk mengukur tingkat keandalan model, dilakukan ekstraksi metrik evaluasi klasifikasi.

```
# 6. PREDIKSI AKURASI
#prediksi akurasi
accuracy = round(accuracy_score(y_test, Y_pred) * 100, 2)
print("Akurasi: ", accuracy, "%")
```

Selain menghitung persentase akurasi keseluruhan, kode menghasilkan *Classification Report* dan *Confusion Matrix*. Evaluasi komprehensif ini penting untuk membedah distribusi kesalahan prediksi, seperti mengidentifikasi jumlah *False Positive* dan *False Negative*.

1.6 Visualisasi Arsitektur Keputusan

Untuk interpretabilitas model, logika keputusan yang dihasilkan oleh algoritma divisualisasikan ke dalam bentuk diagram pohon.

```
# 8. VISUALISASI DECISION TREE (DIREVISI)
# Mengatur ukuran kanvas yang lebih proporsional
plt.figure(figsize=(25, 12))

# Plot tree dengan batasan kedalaman visual (max_depth)
plot_tree(decision_tree,
          feature_names=['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J',
                        'K', 'L', 'M'],
          class_names=['Negative', 'Positive'],
          filled=True,
          rounded=True,
          fontsize=10,          # Ukuran font diperbesar agar terbaca
          max_depth=3)          # KUNCI UTAMA: Hanya menggambar 3 level
teratas

plt.title("Visualisasi Decision Tree (Top 3 Levels)")
plt.tight_layout()

# Menyimpan dengan resolusi tinggi (DPI 300) agar tidak pecah saat di-zoom
plt.savefig("decision_tree_visualization.png", dpi=300, bbox_inches='tight')
plt.show()
print("Visualisasi disimpan sebagai 'decision_tree_visualization.png' dengan
      resolusi tinggi.")
```

Penggunaan parameter `max_depth=3` diterapkan secara strategis untuk membatasi visualisasi hanya pada 3 tingkat percabangan teratas. Hal ini mencegah *visual clutter* akibat dataset yang memiliki 13 dimensi fitur, sehingga aturan bisnis utama tetap dapat diinterpretasikan dengan jelas. Gambar kemudian diekspor dengan resolusi tinggi (DPI 300) agar tidak pecah saat diperbesar.

1.7 Simulasi Prediksi dan Encoding Input

Tahap akhir mendemonstrasikan bagaimana sistem memproses input mentah secara *real-time* untuk menghasilkan prediksi.

```
umur_k = 0
A_k = 0
B_k = 0

if A < 21:
    A_k = 1
if A > 20 and A < 31:
    A_k = 2
```

```
if A > 30 and A < 41:
    A_k = 3
if A > 40 and A < 51:
    A_k = 4
if A > 50:
    A_k = 5
print("kode umur pasien adalah", A_k)

if B == "P":
    B_k = 1
else:
    B_k = 0

if C == "Y":
    C = 1
else:
    C = 0

if D == "Y":
    D = 1
else:
    D = 0

if E == "Y":
    E = 1
else:
    E = 0

if F == "Y":
    F = 1
else:
    F = 0

if G == "Y":
    G = 1
else:
    G = 0

if H == "Y":
    H = 1
else:
    H = 0

if I == "Y":
    I = 1
else:
    I = 0

if J == "Y":
    J = 1
else:
    J = 0

if K == "Y":
    K = 1
else:
    K = 0
```

```

if L == "Y":
    L = 1
else:
    L = 0

if M == "Y":
    M = 1
else:
    M = 0

Train = [A_k, B_k, C, D, E, F, G,
          H, I, J, K, L, M]
print(Train)

test = pd.DataFrame(Train).T

predtest = decision_tree.predict(test)

if predtest == 1:
    print("Pasien Positive ")
else:
    print("Pasien Negative ")

```

Karena model komputasi mensyaratkan input berbentuk numerik, sistem melakukan *Data Binning* (seperti mengonversi rentang umur menjadi kategori kelas angka A_k) dan *Data Encoding* (mengubah input string "Y"/"N" menjadi integer 1/0). Array hasil transformasi ini kemudian diubah menjadi *DataFrame* satu baris untuk dimasukkan ke dalam fungsi `predict()`.